

AIML SESSION 05 ASSIGNMENT SUBMISSION

Name of student: Vaibhav Vijay Kate

Domain/subject name: AIML (Artificial Intelligence & Machine Learning)

College and Branch name: Government Polytechnic College Washim (IF)

Q1. (String Indexing)

Write a program that takes a string input from the user.

Print:

- **The first character**
- **Code:The last character (using negative index)**
- **The 3rd character from the start**
- **The 2nd character from the end**

Add comments explaining positive and negative indexing.

Program Code:

```
#Take a string input from the user
name=input("Enter a string: ")

#The first character of the string can be accessed using index 0 (positive indexing)
print("The first character is:", name[0])

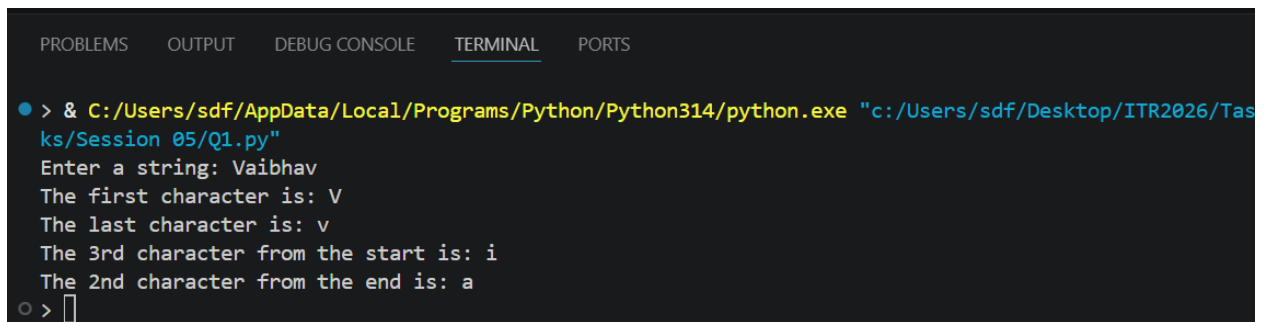
#The last character of the string can be accessed using index -1 (negative indexing)
print("The last character is:", name[-1])

#The 3rd character from the start can be accessed using index 2 (positive indexing)
print("The 3rd character from the start is:", name[2])

#The 2nd character from the end can be accessed using index -2 (negative indexing)
```

```
print("The 2nd character from the end is:", name[-2])
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q1.py"
Enter a string: Vaibhav
The first character is: V
The last character is: v
The 3rd character from the start is: i
The 2nd character from the end is: a
○ > 
```

Q2. (String Slicing)

Take a string input from the user.

Using slicing, print:

- a) The first 5 characters
- b) The last 4 characters
- c) The string in reverse order
- d) Every 2nd character of the string

Include comments explaining the slicing syntax [start:end:step].

Program Code:

```
# Take a string input from the user
name = input("Enter a String:")

# a) The first 5 characters can be accessed using slicing with start index 0 and end index 5
(exclusive)
print("The first 5 characters are:", name[0:5])

# b) The last 4 characters can be accessed using slicing with start index -4 and end index as the
```

length of the string (exclusive)

```
print("last 4 characters are:", name[-4:])
```

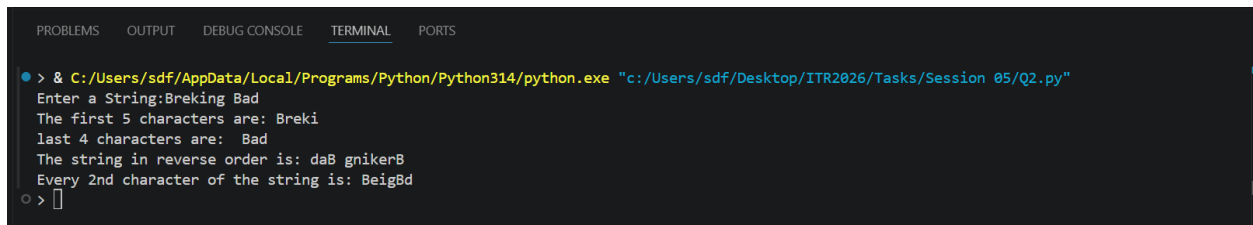
c) The string in reverse order can be accessed using slicing with start index as the length of the string, end index as -1 (exclusive), and step as -1 (to reverse the string)

```
print("The string in reverse order is:", name[::-1])
```

d) Every 2nd character of the string can be accessed using slicing with start index 0, end index as the length of the string (exclusive), and step as 2 (to skip every 2nd character)

```
print("Every 2nd character of the string is:", name[0::2])
```

Output Screenshot :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q2.py"
Enter a String:Breking Bad
The first 5 characters are: Breki
last 4 characters are: Bad
The string in reverse order is: daB gnikerB
Every 2nd character of the string is: BeigBd
○ > 
```

Q3. (String Membership)

Write a program that:

- Takes a main string and a substring as input from the user.
- Checks whether the substring is present in the main string using in and not in.
- Prints appropriate messages (e.g., "Substring found!" or "Substring not found!").
- Note that it is case-sensitive.

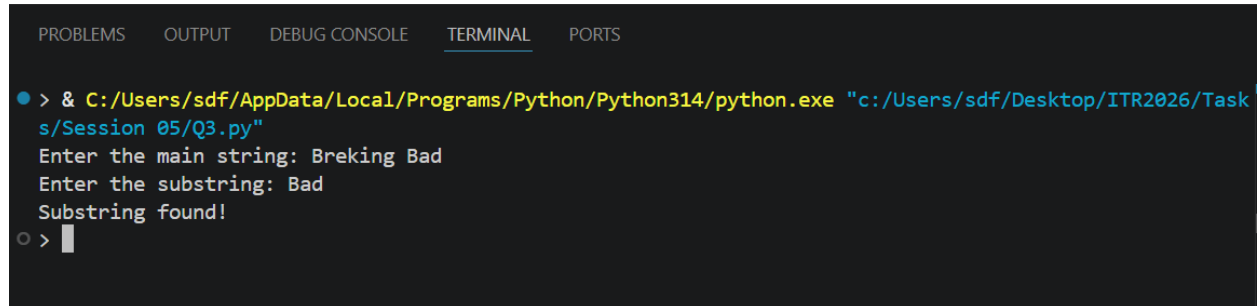
Program Code:

```
# Take a main string input from the user
main_string = input("Enter the main string: ")

# Take a substring input from the user
substring = input("Enter the substring: ")

# Check if the substring is present in the main string
if substring in main_string:
    print("Substring found!")
else:
    print("Substring not found!")
```

Output Screenshot :



The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. The terminal content shows a command prompt where a Python script is executed. The script prompts for a main string and a substring, and then outputs 'Substring found!'.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Task
s/Session 05/Q3.py"
Enter the main string: Breking Bad
Enter the substring: Bad
Substring found!
>
```

Q4. (len() Function)

Write a program that:

- **Takes a string as input.**
- **Prints the length of the string using len().**
- **Prints the last character using len() (without using negative index).**
- **Prints the middle character if the length is odd.**

Add comments on common mistakes with len().

Program Code:

```
# Take a string input from the user
input_string = input("Enter a string: ")

# Print the length of the string using len()

print("The length of the string is:", len(input_string))

# Print the last character using len() (without using negative index)
print("The last character is:", input_string[len(input_string) - 1])

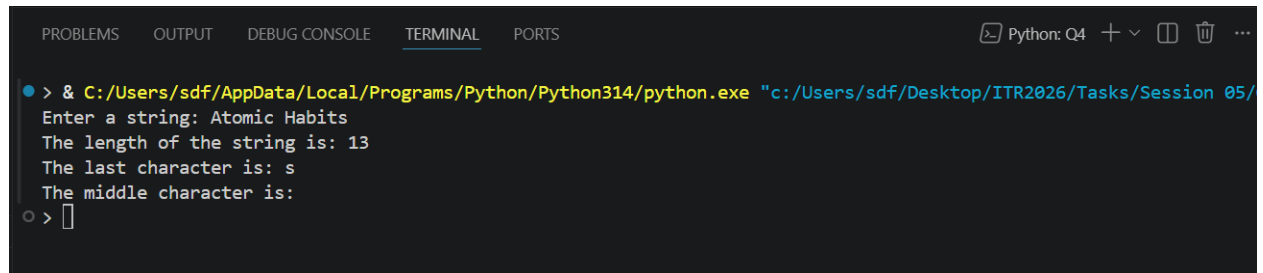
# Print the middle character if the length is odd
if len(input_string) % 2 == 1:
    middle_index = len(input_string) // 2
```

```
print("The middle character is:", input_string[middle_index])
```

else:

```
print("The string has an even length, so there is no single middle character.")
```

OUTPUT Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: Q4 + - [] ...
```

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/  
Enter a string: Atomic Habits  
The length of the string is: 13  
The last character is: s  
The middle character is:  
o > 
```

Q5. (range()) - Basic Forms)

Write a program using for loop and range() to:

a) Print numbers from 0 to 10.

b) Print numbers from 5 to 15.

c) Print odd numbers from 1 to 21.

Explain the three forms of range() in comments.

Program Code:

```
# a) To print numbers from 0 to 10, we can use range() with a single argument which specifies the end of the range (exclusive). The start is by default 0 and the step is by default 1.
```

```
print("Numbers from 0 to 10:")
```

```
for i in range(11): # This will generate numbers from 0 to 10 (11 is exclusive)
```

```
    print(i)
```

```
# b) To print numbers from 5 to 15, we can use range() with two arguments: the start and the end. The start is inclusive and the end is exclusive.
```

```
print("Numbers from 5 to 15:")
```

```
for i in range(5, 16): # This will generate numbers from 5 to 15 (16 is exclusive)
```

```
    print(i)
```

```
# c) To print odd numbers from 1 to 21, we can use range() with three arguments: the start, the
```

end, and the step. The step is set to 2 to get only odd numbers.

```
print("Odd numbers from 1 to 21:")
```

```
for i in range(1, 22, 2): # This will generate odd numbers from 1 to 21 (22 is exclusive)
```

```
    print(i)
```

OUTPUT Screenshot:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Task s/Session 05/Q5.py"
Numbers from 0 to 10:
0
1
2
3
4
5
6
7
8
9
10
Numbers from 5 to 15:
5
6
7
8
9
10
11
12
13
14
15
Odd numbers from 1 to 21:
1
3
5
7
9
11
```

Q6. (range() with Negative Step)

Use range() to print:

- Numbers from 20 down to 10 (decreasing).
- Numbers from 15 down to 1 in steps of 2.

Print each sequence on a new line with proper messages.

Program Code:

```
# a) To print numbers from 20 down to 10, we can use range() with three arguments: the start, the end, and the step. The start is 20, the end is 9 (since it's exclusive), and the step is -1 to decrease the numbers.
```

```
print("Numbers from 20 down to 10:")
```

```
for i in range(20, 9, -1):
```

```
    print(i)
```

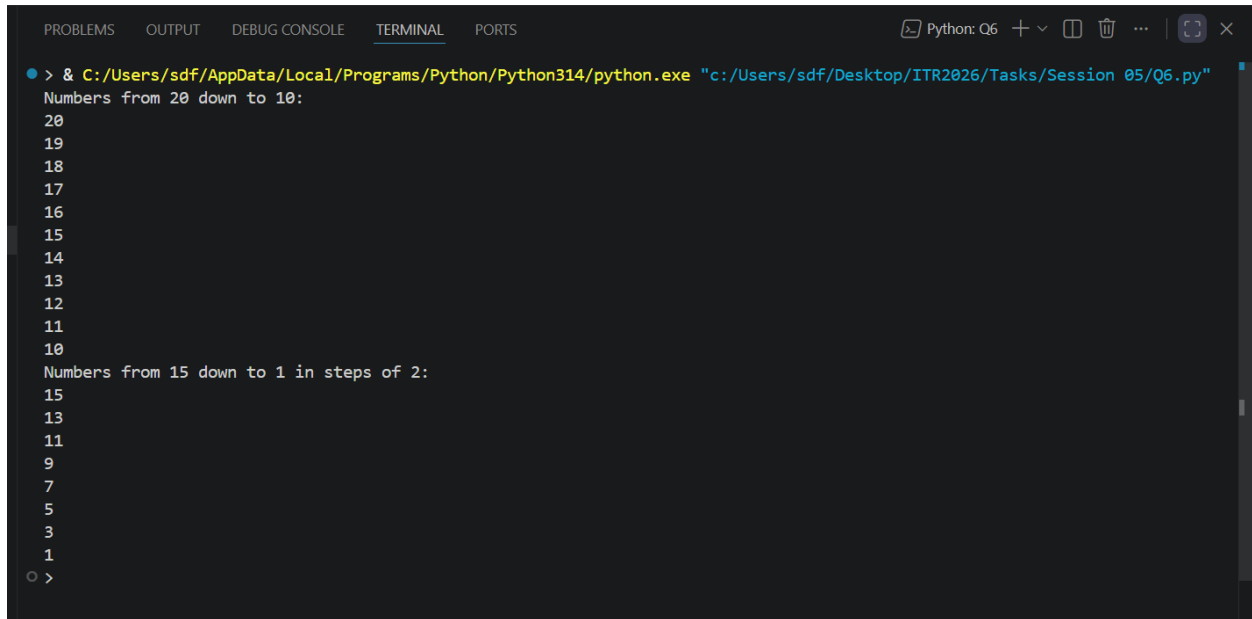
```
# b) To print numbers from 15 down to 1 in steps of 2, we can use range() with three arguments: the start, the end, and the step. The start is 15, the end is 0 (since it's exclusive), and the step is -2 to decrease the numbers.
```

```
print("Numbers from 15 down to 1 in steps of 2:")
```

```
for i in range(15, 0, -2):
```

```
    print(i)
```

OUTPUT Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: Q6 + - [ ] [ ] ... [ ] X
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q6.py"
Numbers from 20 down to 10:
20
19
18
17
16
15
14
13
12
11
10
Numbers from 15 down to 1 in steps of 2:
15
13
11
9
7
5
3
1
○ >
```

Q7. (Combined: Strings + range())

Write a program that takes a string from the user.

Using a for loop and range() with len(), print:

- **Each character of the string one by one (with its index).**
- **The string in reverse order using negative step in range().**

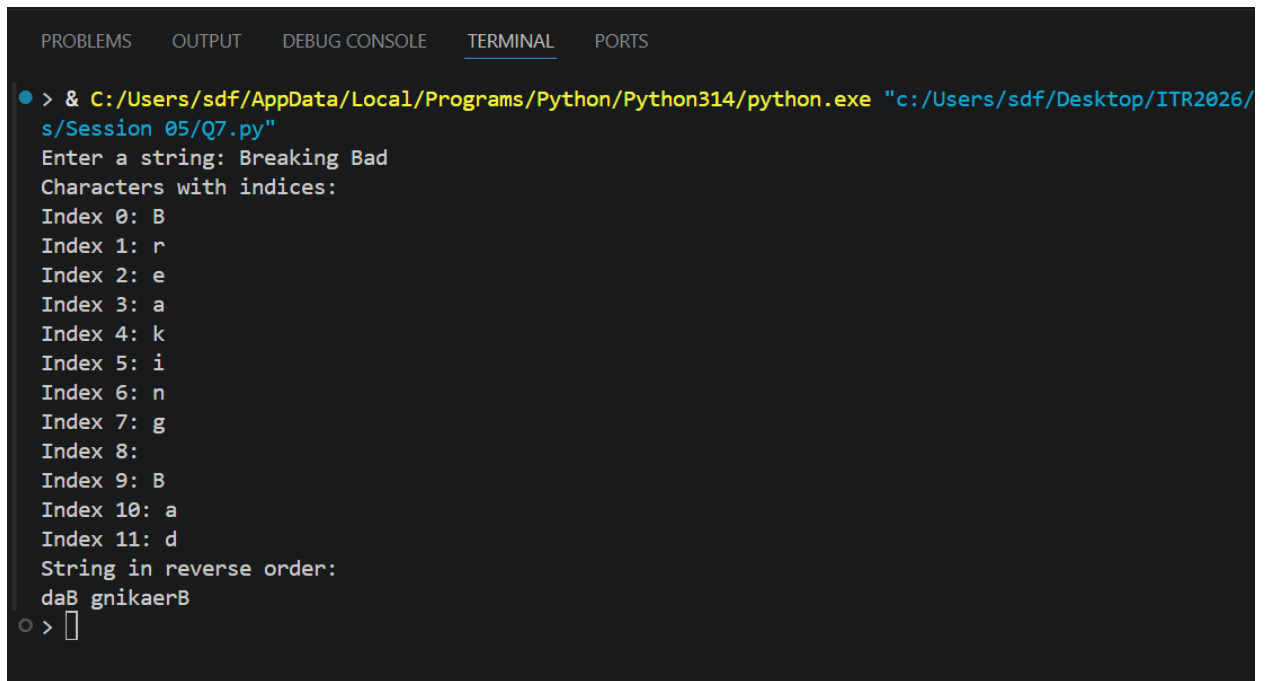
Program Code:

```
# Take a string input from the user
user_string = input("Enter a string: ")

# Print each character with its index
print("Characters with indices:")
for i in range(len(user_string)):
    print(f"Index {i}: {user_string[i]}")

# Print the string in reverse order
print("String in reverse order:")
for i in range(len(user_string) - 1, -1, -1):
    print(user_string[i], end=" ")
print() # Print a newline at the end
```

OUTPUT Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/s/Session 05/Q7.py"
Enter a string: Breaking Bad
Characters with indices:
Index 0: B
Index 1: r
Index 2: e
Index 3: a
Index 4: k
Index 5: i
Index 6: n
Index 7: g
Index 8:
Index 9: B
Index 10: a
Index 11: d
String in reverse order:
daB gnikaerB
○ > 
```

Q8. (Membership with range())

Write a program that:

- Takes a number as input from the user.
- Checks if the number is present in `range(1, 51)` and in `range(10, 100, 5)`.
- Prints clear messages for each check.

Program Code:

```
# Take a number as input from the user
number = int(input("Enter a number: "))

# Check if the number is in range(1, 51)
if number in range(1, 51):
    print(f"{number} is in the range(1, 51).")
else:
    print(f"{number} is not in the range(1, 51).")

# Check if the number is in range(10, 100, 5)
if number in range(10, 100, 5):
    print(f"{number} is in the range(10, 100, 5).")
else:
    print(f"{number} is not in the range(10, 100, 5).")
```

OUTPUT Screenshot:



The screenshot shows a terminal window with a dark background. At the top, there are tabs labeled "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL" (which is selected and underlined), and "PORTS". Below the tabs, a command prompt shows the execution of a Python script. The command is: `> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q8.py"`. The output of the script is: `Enter a number: 12`, `12 is in the range(1, 51).`, and `12 is not in the range(10, 100, 5).`. The prompt `>` is followed by a cursor.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q8.py"
Enter a number: 12
12 is in the range(1, 51).
12 is not in the range(10, 100, 5).
>
```

Q9. (Slicing + range())

Write a program that prints the following using range() and string slicing concepts

where applicable:

- **First 10 even numbers (2 to 20).**
- **Numbers from 1 to 30 in steps of 3.**
- **A string "PythonProgramming" sliced to show "Python", "Programming", and the reverse.**

Program Code:

```
# a) To print the first 10 even numbers from 2 to 20, we can use range() with a step of 2 starting from 2.
```

```
print("First 10 even numbers from 2 to 20:")
```

```
for i in range(2, 21, 2):
```

```
    print(i)
```

```
# b) To print numbers from 1 to 30 in steps of 3, we can use range() with a step of 3 starting from 1.
```

```
print("Numbers from 1 to 30 in steps of 3:")
```

```
for i in range(1, 31, 3):
```

```
    print(i)
```

```
# c) To slice the string "PythonProgramming", we can use string indexing.
```

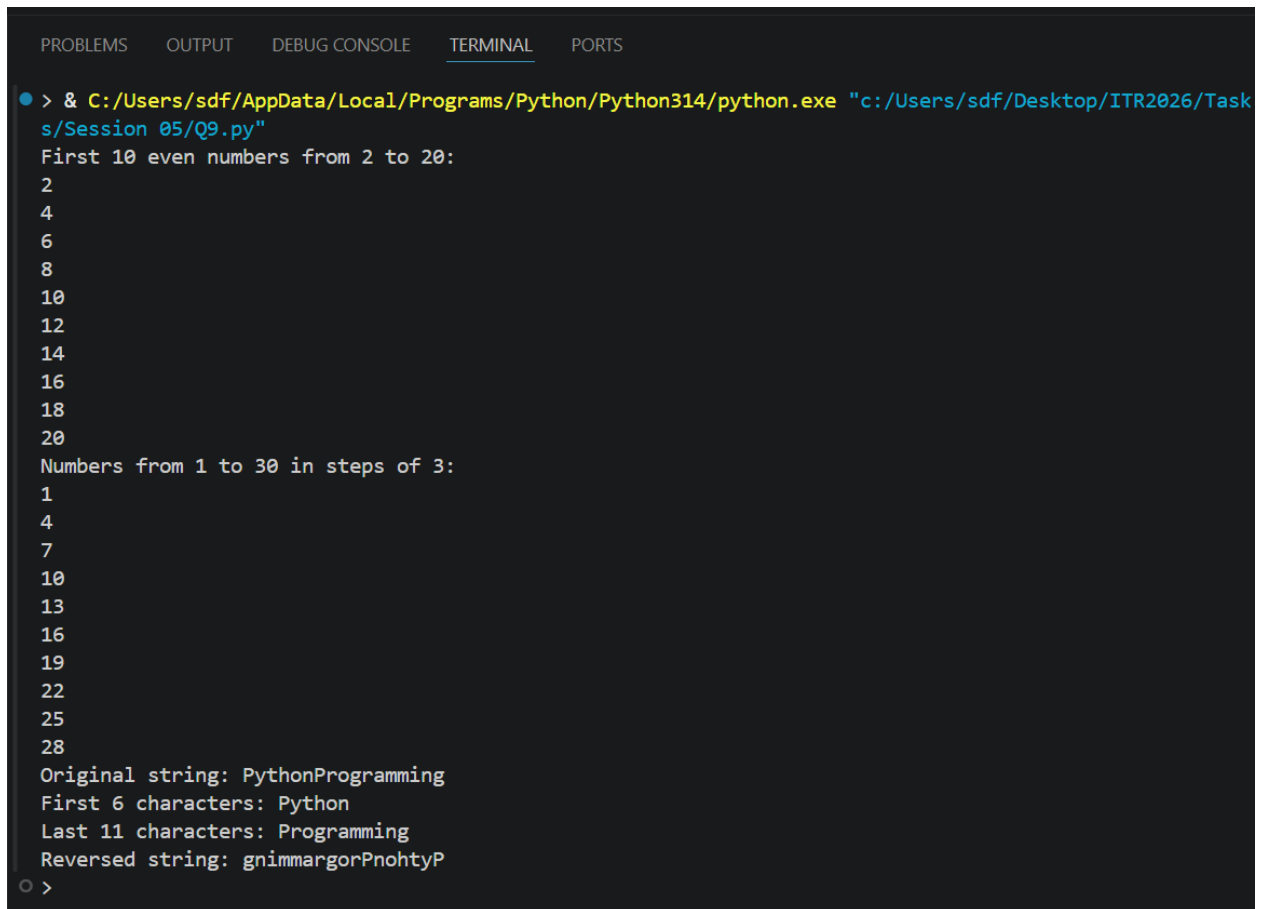
```
user_string = "PythonProgramming"
```

```
print(f"Original string: {user_string}")
```

```
print(f"First 6 characters: {user_string[0:6]}")
```

```
print(f"Last 11 characters: {user_string[6:17]}")  
  
print(f"Reversed string: {user_string[::-1]}")
```

OUTPUT Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q9.py"  
First 10 even numbers from 2 to 20:  
2  
4  
6  
8  
10  
12  
14  
16  
18  
20  
Numbers from 1 to 30 in steps of 3:  
1  
4  
7  
10  
13  
16  
19  
22  
25  
28  
Original string: PythonProgramming  
First 6 characters: Python  
Last 11 characters: Programming  
Reversed string: gnimmargorPnohtyP  
○ >
```

Q10. (Mini Project - Combined)

Create a String Analyzer program:

- Take a string input from the user
- Using functions or directly:

1. Print length of the string.
2. Print first half and second half using slicing.
3. Check if the word "python" (case insensitive — you may convert) is present.
4. Use `range()` to print all characters with their positive and negative indices.
5. Print the string in reverse.

Add proper comments and use meaningful variable names.

Program Code:

```
# Take a string input from the user
input_string = input("Enter a string: ")

# 1. Print length of the string
string_length = len(input_string)
print("Length of the string:", string_length)
```

2. Print first half and second half using slicing

```
half_index = string_length // 2
```

```
first_half = input_string[:half_index]
```

```
second_half = input_string[half_index:]
```

```
print("First half of the string:", first_half)
```

```
print("Second half of the string:", second_half)
```

3. Check if the word "python" (case insensitive) is present

```
if "python" in input_string.lower():
```

```
    print("The word 'python' is present in the string.")
```

```
else:
```

```
    print("The word 'python' is not present in the string.")
```

4. Use range() to print all characters with their positive and negative indices

```
print("Characters with positive and negative indices:")
```

```
for i in range(string_length):
```

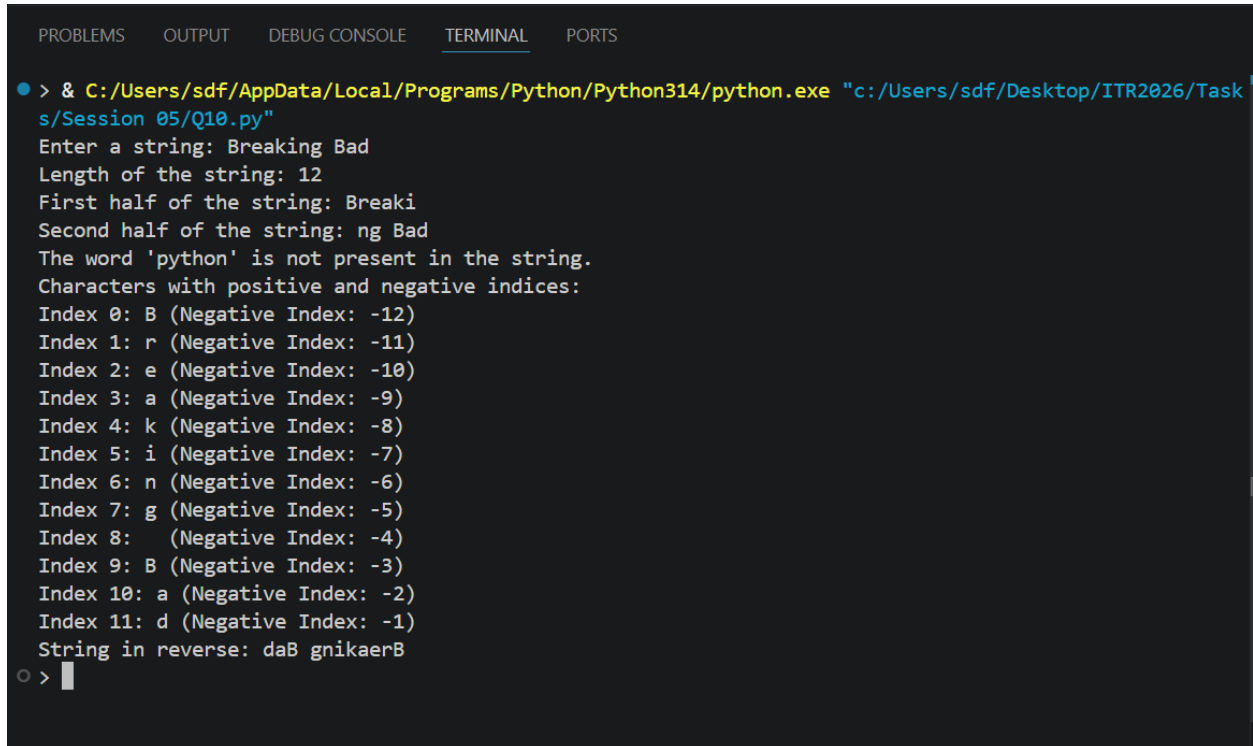
```
    print(f"Index {i}: {input_string[i]} (Negative Index: {i - string_length})")
```

5. Print the string in reverse

```
reversed_string = input_string[::-1]
```

```
print("String in reverse:", reversed_string)
```

OUTPUT Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 05/Q10.py"
Enter a string: Breaking Bad
Length of the string: 12
First half of the string: Breaki
Second half of the string: ng Bad
The word 'python' is not present in the string.
Characters with positive and negative indices:
Index 0: B (Negative Index: -12)
Index 1: r (Negative Index: -11)
Index 2: e (Negative Index: -10)
Index 3: a (Negative Index: -9)
Index 4: k (Negative Index: -8)
Index 5: i (Negative Index: -7)
Index 6: n (Negative Index: -6)
Index 7: g (Negative Index: -5)
Index 8:   (Negative Index: -4)
Index 9: B (Negative Index: -3)
Index 10: a (Negative Index: -2)
Index 11: d (Negative Index: -1)
String in reverse: daB gnikaerB
○ > █
```